

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

Chrono Agentic: Evidence-Grounded Agents for Executable World Simulation

JINGQUAN WANG¹, (Member, IEEE), HONGYU WANG¹, ANDREW NEGRUT², ASHVIN ANILKUMAR¹, RADU SERBAN¹, and DAN NEGRUT¹, (Member, IEEE)

¹University of Wisconsin–Madison, Madison, WI 53706 USA

²Department of Computer Science, Rice University, Houston, TX 77005 USA

Corresponding author: Jingquan Wang (e-mail: jwang2373@wisc.edu).

Jingquan Wang and Hongyu Wang contributed equally to this work.

ABSTRACT *Chrono Agentic* is an inference-time orchestration platform for building executable PyChrono simulations from natural-language, image/edit, simulator-source, and research-question inputs. Around a frozen base model, it maintains task state, selects simulator skills, executes bounded probes, reviews visual and physics evidence, repairs failures, and delivers clean source with replay records. On a matched SimBench slice of 204 Claude-family rows, Chrono Agentic improves average Ref-Doc score from 50.60 to 54.12, a +3.52 point gain. Beyond the benchmark, it rebuilds EXUDYN and MuJoCo/Menagerie material as Chrono artifacts with source cards, runs, captures, diagnostics, and equivalence notes. Its auto-research workflow turns pendulum questions into equations, predicates, PyChrono probes, search traces, repairs, and stored results. In Scenario I, top-down, wide-site, and trench-side camera views provide visual evidence; benchmark and physics statements remain tied to source, run, and diagnostic records. The paper evaluates simulator agents by executable source, replay evidence, and stated scope.

INDEX TERMS Large language models, PyChrono, simulation agents, inference-time orchestration, world simulation.

I. INTRODUCTION

Simulator generation differs from ordinary Python generation. A PyChrono digital twin is executable code and a scientific model: it must instantiate physical state, constraints, contacts, solvers, timesteppers, sensors, and visualization loops using Project Chrono APIs [1]. A script can import successfully while still encoding the wrong world; acceptance requires a replayable artifact with evidence.

Two Chrono-centered studies motivate this work. ChronoLLM represents a training-time adaptation path: simulator-specific examples and virtual-experiment scripts improve the model prior for PyChrono assistance [2]. The Coding Agent as World Simulator study explores the complementary inference-time orchestration path: planning, code generation, execution, visual review, and physics analysis for explicit simulator programs [3]. Chrono Agentic develops that second path as an external orchestration platform around a frozen base model.

Chrono Agentic is a platform for inference-time simulator construction, not a new trained model. It tests whether a frozen LLM becomes a more reliable generator of runnable simulator artifacts when task state, simulator-specific skill routing,

bounded execution, multimodal review, physics diagnostics, and verification-gated repair are added at inference time. The loop draws on tool-using, self-refining, and executable-code agents [4], [5], [6], [7], [8], [9], but its action and observation spaces are specific to multibody and multiphysics simulation.

Many simulator failures are invisible to ordinary code benchmarks. A generated script may compile while omitting collision, freezing a body that should move, placing a joint frame in the wrong coordinate convention, using a visually plausible but physically wrong constraint, or delivering a plot whose trace was produced by a broken stepping loop. These errors change the scientific content of the artifact. A simulator agent needs a stricter contract than “write Python that runs.” It must keep the user’s intended world, the simulator realization, the review evidence, and the final deliverable separated enough that a reader can replay the result and check the claim.

This paper evaluates the platform across three pipeline interfaces. A matched SimBench benchmark evaluation compares direct single-LLM generation with Chrono Agentic on the same system-turn tasks and shows average Ref-Doc score improving from 50.60 to 54.12, a +3.52 point gain. Scenario I treats natural-language, image, sketch, and edit requests as

inputs for runnable PyChrono source with separated execution, visual, physics, repair, and clean-delivery evidence; its top-down, wide-site, and trench-side camera views provide visual evidence, not benchmark or physics results. Scenario II rebuilds EXUDYN and MuJoCo/Menagerie material as Chrono artifacts with source cards, diagnostics, captures, and explicit equivalence limits. Scenario III turns pendulum research questions into equations, predicates, PyChrono searches, convergence traces, predicate repairs, and stored result records. The accepted artifact is the unit of analysis; prompts and benchmark rows are supporting evidence channels.

The paper contributes a PyChrono generation platform, a matched SimBench benchmark result, a cross-simulator reconstruction workflow with explicit equivalence boundaries, and an auto-research trace showing how a research question is turned into predicates, probes, repairs, and accepted evidence.

Two questions organize the study. The engineering question is whether inference-time orchestration can turn user intent or source-simulator material into PyChrono artifacts that compile, run, and carry enough evidence for review. The evaluation question is how to report such artifacts without overstating what a static benchmark score, a video frame, a source-card reconstruction, or an auto-research result proves. The main text reports the score, transfer, and research evidence; the appendix preserves one representative Scenario II code excerpt and a normalized auto-research trace.

II. RELATED WORK

Chrono Agentic connects work on world models, simulator systems, and executable code agents. Its output is runnable PyChrono source plus evidence, rather than a video rollout, a unit-test-passing snippet, or a newly trained simulator-specific model.

World models, embodied simulators, and scene generation.

World-model research studies learned latent or generative dynamics for control, interaction, and visual prediction [10], [11], [12], [13], [14], [15]. Recent video-world-model benchmarks ask whether generated video obeys physical regularities [16], [17]. Embodied AI simulators and scene-generation systems provide interactive or procedurally generated environments for agents [18], [19], [20], [21], [22], [23], [24]. These lines motivate the interface studied here, but the artifact in this paper remains explicit Chrono source plus replay evidence. A video, mesh preview, or simulator asset can be part of the evidence record; it is not by itself the accepted scientific object.

Executable code agents. Code-generation benchmarks and software-engineering agents provide the outer plan–act–observe–repair loop [25], [26], [4], [5], [6], [7], [8], [9], [27]. Chrono Agentic keeps this loop but changes the acceptance signal. A repository patch is often judged by tests; a simulator artifact must also preserve a physical world. Unit tests and imports are insufficient: the accepted state must include execution logs, visual frames or videos when the task has a visual claim, numerical diagnostics when the task has a physical invariant, repair history when failures occur, and a stated boundary on what the artifact does not prove.

LLM physical reasoning and simulator-specific assistance.

Related language-model systems construct physical scenes or embodied tasks [28], [29], and training-time adaptation can improve the model prior for PyChrono. ChronoLLM shows that PyChrono assistance benefits from simulator-specific examples, current API knowledge, and full virtual-experiment scripts [2]. The Coding Agent as World Simulator study develops the broader inference-time orchestration idea of using coding agents as explicit simulator builders [3]. Chrono Agentic assumes a frozen base model and studies the external orchestration, memory, skill routing, execution, visual/physics review, repair, and clean-source delivery needed to make that model’s simulator claims checkable.

Evidence and reporting standards. Simulation verification and validation emphasize that a model claim is only as strong as the evidence connecting implementation, scenario, and result [30]. Datasheets and model cards motivate structured reporting of artifacts, assumptions, and intended use [31], [32]. This paper adapts that reporting discipline to agent-produced simulators: the source file, run evidence, visual captures, diagnostics, repair notes, and result tables are stored separately so that benchmark scores, transfer demonstrations, and auto-research results do not silently upgrade one evidence type into another.

III. PYCHRONO DIGITAL-TWIN AGENT SETTING

A. PROJECT CHRONO AND PYCHRONO

Project Chrono is an open-source multi-physics simulation engine for multibody dynamics, finite elements, vehicles, granular terrain, robotics, sensors, and related applications [1]. Through PyChrono, a digital twin must specify a system, materials and collision, bodies or meshes, joints or contacts, solver/timestepper choices, visualization or sensors, and a stepping loop. The idiom changes across vehicles, FEA, robotics, sensors, and contact-rich mechanisms.

This creates a high-dimensional error surface for LLMs: the code may import while using the wrong module, version, topology, sensor update, or solver idiom. ChronoLLM addresses the model-side adaptation problem: PyChrono assistance depends on current API knowledge and complete virtual-experiment scripts rather than isolated fragments [2]. Chrono Agentic addresses the process-side verification problem with typed task memory, skills, execution feedback, visual/physics review, and repair before accepting a generated artifact.

The setting also has several input types. A user may ask for a pendulum in prose, upload a sketch of a linkage, request an edit to an existing mechanism, provide a source file from another simulator, or pose an inverse-design question. These inputs are heterogeneous, but the simulator obligations are similar: the agent must determine which objects exist, which ones are fixed or dynamic, how they are connected, which physical quantities are to be measured, which quantities are merely visual, and what evidence would count as success. For that reason, the paper treats Chrono Agentic as a platform for producing reviewable artifacts rather than as a prompt template.

The implementation target in this manuscript is PyChrono 9.0.0. That detail matters because PyChrono examples depend on version-specific modules, visualization APIs, vehicle and sensor namespaces, and parser availability. For a simulation-agent paper, “runs locally” is not a vague convenience condition; it is part of the artifact definition. The clean deliverable is the runnable source, while supporting evidence can include logs, frames, videos, numeric traces, source cards, and repair notes that are kept outside the final script.

IV. METHODOLOGY: BUILDING CHRONO AGENTIC

A. AGENT CONSTRUCTION CONTRACT

The method is named *Chrono Agentic*. It is an inference-time orchestration platform specialized for PyChrono 9.0.0, combining the simulator-specific setting of Project Chrono with the execution-and-repair pattern of code agents [1], [7], [8]. The method boundary is the artifact: the agent maintains explicit state over generation, execution, review, repair, and accepted outputs. Benchmarks are consulted only after artifacts exist. Table 1 states the method contract.

TABLE 1: Chrono Agentic method contract, grounded in Project Chrono and executable-code-agent workflows [1], [7], [8].

Inputs	Accept natural-language, image/sketch/screenshot, edit, simulator-source, or research-question task cards.
State	Maintain a compact task card, selected PyChrono skill context, artifact state, evidence digest, and repair ledger.
Loop	Plan, draft source, run bounded PyChrono probes, inspect visual/physics evidence, then repair or accept.
Delivery	Return clean PyChrono source plus separated logs, frames, traces, diagnostics, verdicts, and result records.

Under this contract, importable Python is only the starting point. Scenario gates require execution or static justification before acceptance. Review channels remain separate from clean delivery: frame capture, videos, numeric summaries, visual feedback, and physics verdicts can repair the artifact, while the accepted source can still be exported as a clean simulator file when a benchmark or evaluation setting requires it. Scalar statements cite stored evidence. If a transfer artifact imports geometry without reproducing source-simulator dynamics, the scope is stated. If an auto-research run searches a finite-time predicate rather than an analytic separatrix, the predicate is stated.

B. PIPELINE AND EVIDENCE DESIGN

The evaluation uses three pipeline settings. Scenario I converts natural-language, image, sketch, or edit intent into executable Chrono simulation code. SimBench supplies the text-only benchmark result for this interface, and the construction-site example supplies top-down, wide-site, and trench-side camera views of the generated scene. Scenario II reconstructs EXUDYN and MuJoCo-style material as PyChrono artifacts. Scenario III asks whether research questions can become equations, predicates, PyChrono searches, witness runs, convergence gates, and stored records. Table 2 defines the accepted evidence unit for each scenario.

TABLE 2: Scenario evidence units.

Scenario	Input or role	Accepted evidence unit
Scenario I	Natural language, image, sketch, screenshot, or edit request	Clean PyChrono source plus run, visual, physics, and repair evidence; SimBench evaluates the corresponding text-only benchmark submission, while top-down, wide-site, and trench-side camera views support visual review only.
Scenario II	EXUDYN scripts and MuJoCo plus Menagerie-style robot assets	Source card, reconstructed PyChrono artifact, bounded no-visualization run, visual capture when applicable, diagnostics, and explicit equivalence boundary.
Scenario III	Research question over a pendulum model	Equation stack, operational predicate, PyChrono probe suite, search trace, witness replay, convergence and sensitivity checks, and stored result record.

A single benchmark score does not cover the engineering question studied here: the agent must know when to construct a simulation artifact, run it, inspect it visually, compare it with physics diagnostics, patch it, and narrow the reported scope. The three pipeline scenarios expose those obligations at different levels of difficulty. The construction-site views in Scenario I show how views from different camera viewpoints can be preserved without being promoted beyond their evidence channel.

Each evidence channel supports a different statement. A SimBench row supports a benchmark score for the source submitted to the judge. An execution log supports run success for a stated environment. A visual frame or video supports framing and scene-behavior claims when attached to a generated scene. A physics diagnostic supports a mechanism-specific invariant or predicate. A transfer source card supports semantic reconstruction rather than bit-exact simulator equivalence. Dynamics-validity statements require source, run, and diagnostic evidence; a benchmark score alone cannot establish visual or physical adequacy.

Figure 1 summarizes the generic Chrono Agentic architecture instantiated by the following state, loop, and evidence-interface subsections. Scenario-specific figures later in the paper show concrete pipeline instances.

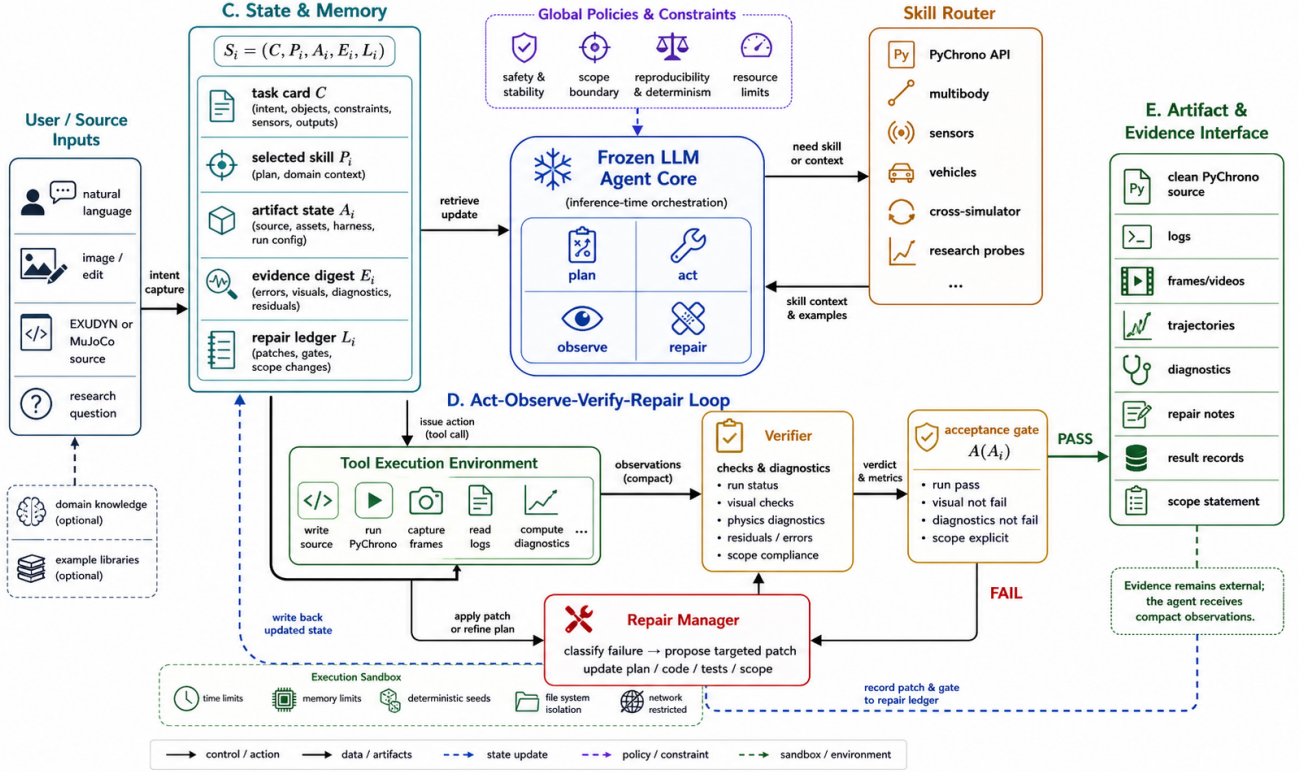


FIGURE 1: Generic Chrono Agentic architecture for Sections C–E. A frozen LLM agent core plans, acts, observes, and repairs over compact external state; a skill router supplies PyChrono contexts; tool execution produces simulator observations; a verifier and repair manager close the acceptance loop; and the evidence interface exports clean source separately from logs, frames, trajectories, diagnostics, repair notes, result records, and scope statements.

C. AGENT STATE AND MEMORY

The orchestration layer maintains bounded state instead of a long conversational transcript. At iteration i , the external state is

$$S_i = (C, P_i, A_i, E_i, L_i), \quad (1)$$

where C is the resettable task card, P_i is the current plan and selected PyChrono skill context, A_i is the source/assets/run-harness artifact state, E_i is the compact evidence digest, and L_i is the repair ledger. Raw frames, videos, logs, and numeric traces remain in the external evidence store; model calls see only the compact state card and write an updated plan, code patch, or accept/repair decision. This replaces raw transcript memory and prevents review evidence from becoming hidden evaluation content [8], [27].

The task card C stores the simulator-facing intent: objects, masses or inferred defaults, coordinates, constraints, contacts, terrain, sensors, visualization requirements, and requested outputs. The plan P_i records the selected PyChrono domain context, for example a multibody mechanism context, a sensor context, a vehicle context, or a cross-simulator reconstruction context. The artifact state A_i points to the current source file, assets, run harnesses, and result records. The evidence digest E_i summarizes only what the next model call needs: traceback class, missing API, visible geometry mismatch, failed physical

predicate, non-finite state, or accepted diagnostic. The repair ledger L_i records what was changed and why.

The state tuple in Eq. (1) is intentionally compact. It prevents two common failure modes in long agent sessions: carrying stale code assumptions after a repair, and treating the entire conversation as evidence. Evidence remains external and replayable; the model receives only the distilled observations needed to make the next patch.

D. GENERATION, REVIEW, AND DELIVERY LOOP

Chrono Agentic treats one turn as a verification-gated source-production loop. The gates combine PyChrono API discipline, execution feedback, visual review, and physics repair [1], [2], [33], [34], [35], [6]. In compact form, the loop is: intent card $\rightarrow$ source draft $\rightarrow$ bounded run $\rightarrow$ visual/physics review $\rightarrow$ clean-source delivery. Failed import, runtime, camera, topology, motion, contact, residual, or diagnostic checks return compact evidence to the task state and trigger another patch.

The acceptance decision is scenario-specific but has a common structure. Let $R(A_i)$ denote the bounded execution result, $V(A_i)$ the visual review when the task has a visual claim, $D(A_i)$ the numerical or physics diagnostics, and $B(A_i)$

the declared scope. The artifact is accepted only when

$$\mathcal{A}(A_i) = 1[R(A_i) = \text{pass} \wedge V(A_i) \neq \text{fail} \wedge D(A_i) \neq \text{fail} \wedge B(A_i) \text{ explicit}]. \quad (2)$$

Equation (2) is a gate pattern rather than a scalar score. Some artifacts are headless-only and have $V(A_i) = \text{not applicable}$, while visual transfer artifacts require a nonblank, framed, semantically recognizable capture. Each accepted artifact has a declared acceptance path rather than an unrecorded visual judgment.

When a gate fails, the patch is targeted. Import errors map to API or module repair; runtime errors map to the owning PyChrono object or native binding; visual errors map to camera, geometry, material, or scene-scale repair; physics errors map to topology, constraints, contact material, forces, or timestep choices. The agent does not lower the gate to preserve progress. It either repairs the artifact, narrows the claim, or records the blocker.

E. EVIDENCE INTERFACE

Chrono Agentic produces an evidence record in addition to a final source file. The source channel stores the clean deliverable; execution, visual, physics, repair, and research channels store logs, frames, trajectories, diagnostics, patches, and result ledgers [31], [32]. The model sees a compact digest, while the full evidence remains replayable outside the final source.

The paper does not collapse all artifacts into one table because the evidence channels differ. A prompt-to-world-code task may produce executable source, a video, a physics verdict, and a benchmark source submission. An EXUDYN port may produce a source-card comparison, a no-visualization run, an image capture, and mechanism-specific diagnostics. An auto-research case may produce a search trace, a witness pair, an equation-derived baseline, and a result record. These evidence types all use the acceptance structure in Eq. (2).

V. SCENARIO I: INTENT AND IMAGE TO CHRONO WORLD CODE

Interface claim. Scenario I asks whether a user-facing intent channel can become executable Chrono simulation code. The input may be natural language, an image/sketch/screenshot, or an edit request. The accepted output is a clean PyChrono source file that instantiates bodies, constraints, contacts, solver/timestep choices, sensors, visualization, and diagnostic evidence. The target is a reviewable simulation artifact with replay evidence, not merely Python text.

The Scenario I task begins with a task card rather than immediate code. For a natural-language request, the card extracts the physical objects, support relations, desired motion, outputs, and missing quantities that can be safely defaulted. For an image, sketch, or screenshot, the card also records visible geometry, camera viewpoint, contact or support cues, and ambiguities that must be resolved before coding. For an edit request, the card identifies which part of the existing source is preserved, which part changes, and which previous

evidence remains valid. This card prevents a common failure in simulator generation: a visually rich answer that silently drops the physical relation that made the user's request meaningful.

A. CLOSED-LOOP SCENE-GENERATION PIPELINE

Figure 2 summarizes the agent roles used for this Scenario I interface before the acceptance checks are applied.

The Scenario I pipeline is a closed-loop construction workflow, not a single prompt-to-code call. A plan agent first converts the user request and optional image into a structured construction plan. The plan records the high-level scene intent before source code is written: coordinate convention, fixed and movable objects, collision expectations, asset families, requested cameras, expected outputs, and any reproducibility requirements. A code agent then translates the approved plan into PyChrono source with access to simulator skills, reusable scene assets, file inspection, code search, and shell execution.

The code agent receives execution feedback, but acceptance is separated from generation. The generated script is run, simulation data are passed to a review agent, and rendered video or frames are passed to a visual-analysis agent. The visual-analysis agent converts observed behavior into a structured description for review. If execution or semantic review fails, the review agent returns a physical and visual report to the code agent; the code agent revises the source and reruns the scene. If the scene passes, the loop stops and the artifact is split into clean source plus evidence.

The generated PyChrono source follows a source-plus-review pattern. The simulator core instantiates the model with explicit coordinate constants, materials, bodies, joints, contacts, loads, sensors, timesteps, and visualization setup. Review scaffolding may capture frames, videos, numeric traces, and diagnostic summaries, but it is kept separate so that a benchmark can compare clean source submissions. This separation lets the review harness judge and repair the model without changing the physical content being evaluated. Table 3 lists the Scenario I acceptance checks.

TABLE 3: Scenario I acceptance checks.

Gate	Scenario I acceptance check
Import/runtime	The script imports PyChrono modules under the target environment and advances for a bounded duration without native crashes or non-finite state.
World topology	Bodies, fixed supports, links, contacts, loads, sensors, and terrain match the task card rather than only matching a plausible visual scene.
Visual evidence	The capture shows the requested objects in frame, with motion or placement consistent with the prompt or image.
Physics evidence	Diagnostics confirm nontrivial motion, contact, constraint, sensor, or output behavior as required by the task.
Clean delivery	Review-only recording and analysis scaffolding can be removed while preserving the simulator core.

B. SIMBENCH BENCHMARK RESULT

SimBench contains 34 systems across three turns, producing 102 code samples per condition [36]. Here we report only the average Ref-Doc gain; the SimBench paper gives score details.

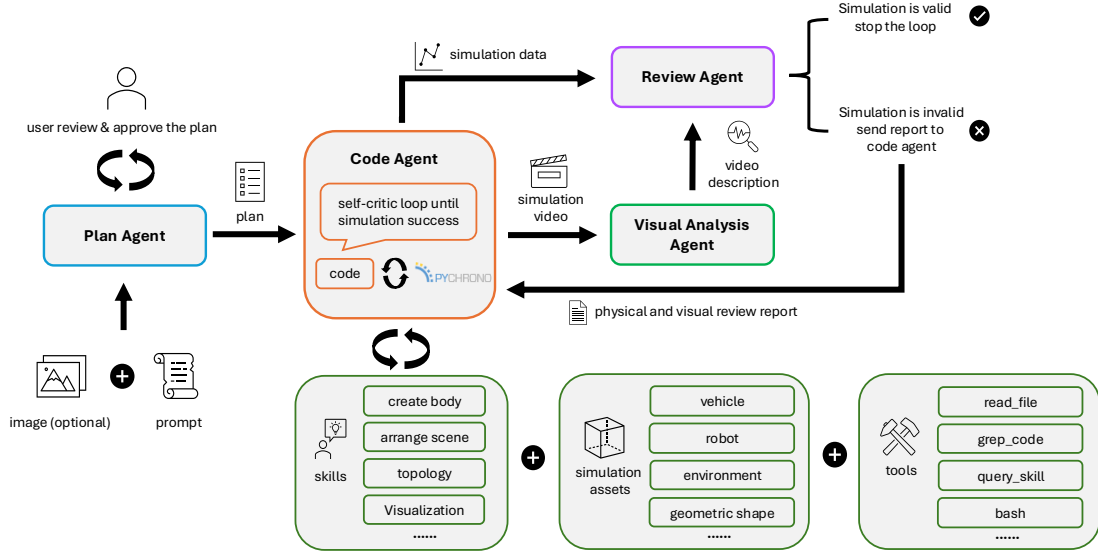


FIGURE 2: Scenario I prompt/image-to-PyChrono scene-generation pipeline. The plan agent turns the user prompt and optional image into an approved construction plan covering scene intent, assets, cameras, and reproducibility requirements. The code agent implements and reruns the PyChrono scene with simulator skills, reusable assets, and development tools. Simulation data and rendered video are separated into physical review and visual-analysis channels; the review agent accepts valid simulations or returns a structured visual/physics repair report to the code agent for another revision cycle.

TABLE 4: Scenario I SimBench average Ref-Doc score.

Condition	Score
Direct prompt-to-code	50.60
Chrono Agentic deliverable	54.12
Δ	+3.52

On this benchmark, Chrono Agentic improves the average Ref-Doc score from 50.6 to 54.1, a **+3.5** point gain. We defer score construction and interpretation details to SimBench itself [36].

C. CONSTRUCTION-SITE SCENE DEMONSTRATION

The main Scenario I example is a construction-site scene generated from a human-authored request. It is presented as a generated-scene record rather than as a SimBench row: the prompt becomes a task plan, the plan becomes PyChrono source, the source produces a drivable scene, and the accepted artifact carries top-down, wide-site, and trench-side camera views. The example shows the Scenario I evidence format without treating the views as benchmark rows or physics-validation results.

The main example is a construction-site driving scene rather than a minimal geometric prompt. The prompt block below is written as a human scenario request, not an implementation checklist. It specifies site scale, a coordinate convention, fixed-versus-movable object semantics, collision stability, no-overlap placement, body metadata, scripted vehicle motion, and video output. Those details matter because a generated

scene can look plausible while violating the user's physical contract, for example by fixing all props, omitting collision shapes, placing the truck in overlap, or recording no reproducibility information.

Construction-site Scenario I prompt.

Create a construction site driving scene.

The site should be about 60 m by 38 m, and set +Z as the up direction.
Make the ground the only fixed object. All construction props should be movable, collidable bodies.

Populate the site with realistic construction clutter assets, pick from open-license assets where possible, with simple collision shapes so the simulation remains stable.

Place a six-wheel work truck in an open lane with no initial overlaps. Record the random seed, check for object overlaps before the run, and report each body's mass, approximate shape, and pose.
Drive the truck down the lane with a scripted sequence of throttle, steering, and braking, and save a video of the run.

The generated artifact follows those high-level constraints while leaving detailed site composition to the agent. The scene contains a box truck, trench corridor, pipes, barriers, stacked materials, construction equipment, boundary markers, and viewpoint-specific cameras around the same site. The

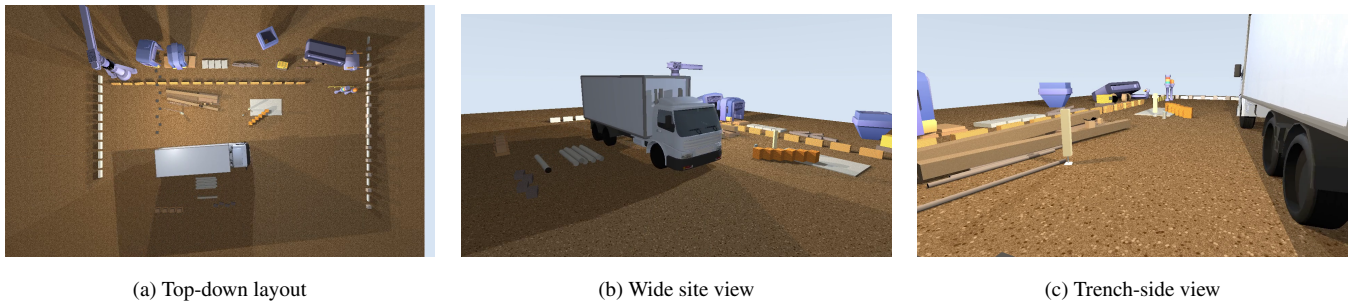


FIGURE 3: Construction-site Scenario I demonstration. The preserved 3 s top-down, wide-site, and trench-side views show global layout, vehicle/equipment context, and low camera placement near trench-side construction details.

example is more demanding than a single attractive screenshot. A single view can hide whether the layout is coherent; the top-down view checks global placement, the wide-site view checks recognizable scene composition around the vehicle, and the trench-side view checks low camera placement near pipes, barriers, and the vehicle path.

Figure 3 provides visual evidence and is not used as a score table. The top-down view exposes the truck pose relative to the lane, trench boundary, equipment clusters, and marked site perimeter. The wide-site view shows the six-wheel truck in an open region with nearby pipes, barriers, stacked material, and equipment. The trench-side view places the camera near the work zone and truck body, making close-range occlusion and camera placement visible. Together, the views reduce dependence on a single favorable camera angle.

Scenario I separates two evidence channels. SimBench gives a matched benchmark result; the construction-site demonstration gives a detailed generated-scene artifact. The demonstration supports a specific statement: Chrono Agentic can preserve a human prompt, source-level construction constraints, synchronized top-down, wide-site, and trench-side camera views without turning those views into benchmark or dynamics-validation results.

VI. SCENARIO II: CROSS-SIMULATOR TRANSFER INTO PYCHRONO

The second scenario asks whether the same agent workflow can reconstruct source-simulator material in PyChrono. The accepted object is a verified transfer artifact: source review, source card, reconstruction, bounded execution, visual capture, diagnostics, repair notes, and a clean runnable script. We separate replay evidence from claims of bit-exact simulator equivalence.

Cross-simulator transfer is harder than prompt-to-code generation because the source simulator's API does not map one-to-one to PyChrono. EXUDYN examples use object types, markers, connectors, loads, sensors, and plotting conventions that differ from Chrono. MuJoCo and Menagerie assets expose MJCF bodies, geoms, joints, meshes, materials, defaults, and actuator conventions rather than Chrono bodies and links. Chrono Agentic treats transfer as semantic reconstruction: it preserves the physical mechanism, visible state, control phase,

and diagnostics that are portable, and it names the parts that are approximate, unsupported, or outside the reported scope.

The transfer workflow has four stages. First, the source card records what the original example appears to model: bodies, constraints, loads, actuation, sensors, assets, parameters, expected diagnostics, and nonportable features. Second, the agent writes a PyChrono reconstruction using native Chrono objects or an explicitly labeled semantic surrogate. Third, the port is compiled and run in a bounded no-visualization mode, with diagnostics chosen for the mechanism rather than only for generic success. Fourth, visual captures are regenerated when the port has a visual component, and the clean source plus evidence records are preserved.

Porting trace and EXUDYN ports. One preserved session audited 345 EXUDYN source files, produced 2,018 visible messages and 164 accepted checkpoints, and accepted ports only after a compact contract: source-card reconstruction before code, import and bounded execution before acceptance, visual and mechanism-specific diagnostics before promotion, and separated provenance after acceptance.

The EXUDYN portion is a traceable porting study rather than a set of isolated examples. Each candidate starts with an inventory classification. Direct candidates can be represented by Chrono bodies, joints, loads, or FEA objects with limited approximation. Partial candidates preserve the mechanism but require a documented simplification. Non-direct or review candidates are not forced into unsupported exactness; they become source-card evidence until the missing Chrono-side support is clear. This triage keeps the evidence readable: a successful visual capture does not by itself prove physical equivalence, and a nonportable source feature is recorded instead of hidden.

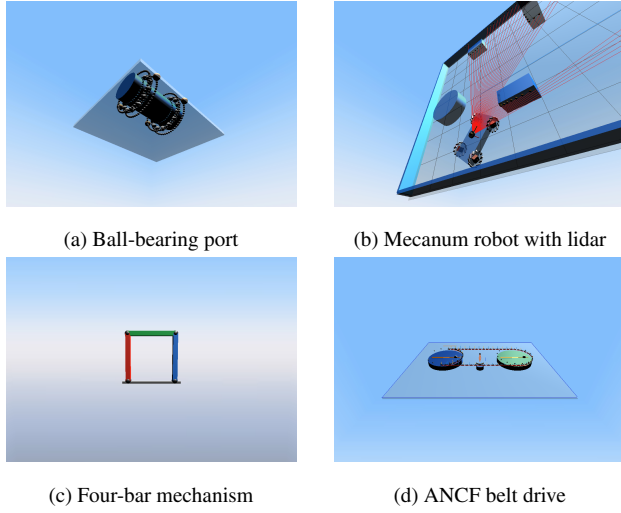


FIGURE 4: EXUDYN-to-PyChrono transfer witnesses. The four panels are direct visual captures from accepted ports; each panel supports a source-card and bounded-run evidence record rather than a claim of bit-exact EXUDYN equivalence.

A. MUJOCo/MENAGERIE ARTIFACTS AND ROBOT DYNAMICS

The Menagerie portion separates asset portability from dynamics reconstruction. Chrono Agentic imports MJCF-side meshes, materials, body trees, and parsed joint declarations, then rebuilds selected mechanisms as Chrono-native dynamics witnesses with bodies, constraints, actuation or contact, bounded runs, and diagnostics. Figure 5 keeps these two evidence types separate.

The first Menagerie pathway is asset import. The importer resolves mesh references, material colors, nested body transforms, simple primitives, and selected MJCF defaults, then lowers them into Chrono visual geometry. This path answers a structural question: can the source asset be inspected in a PyChrono scene with recognizable body hierarchy and visual identity? It does not answer whether the same contacts, actuators, controllers, or MuJoCo solver behavior have been reproduced.

The second pathway is selected dynamics reconstruction. Here the agent writes Chrono-native witnesses such as a torque-driven 2R arm, a skid-steer contact base, and a compliant quadruped stance model. These witnesses use Chrono bodies, joints, motors, contact materials, spring-damper elements, and diagnostics to show that the relevant mechanism can be expressed and checked in PyChrono. They are narrower than the original source ecosystem: the claim is not a wholesale MuJoCo reproduction but a validated Chrono dynamics reconstruction for a stated mechanism.

Scope statement. Imported MJCF assets establish portability of mesh, material, body-tree, and parsed-joint structure. The 2R arm, skid-steer, and compliant-stance witnesses establish Chrono-native dynamics reconstruction, not bit-exact MuJoCo equivalence; each script compiles, runs with `-no-vis`, and emits mechanism-specific diagnostics.

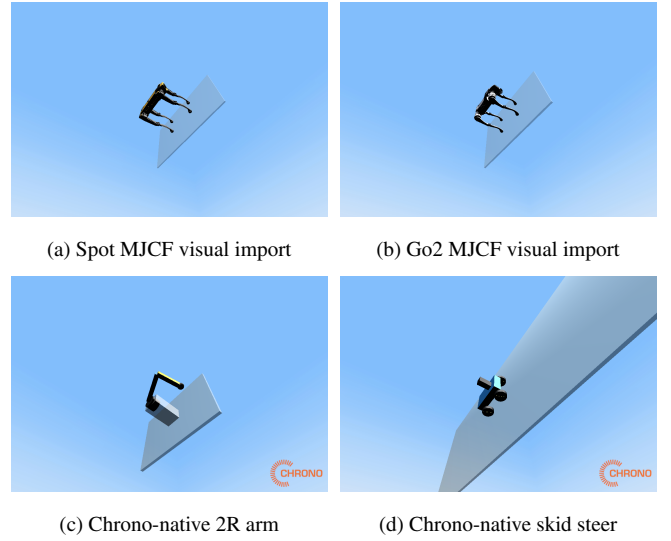


FIGURE 5: MuJoCo/Menagerie transfer witnesses. The top row shows imported MJCF-side visual assets; the bottom row shows selected Chrono-native dynamics witnesses used for mechanism-specific validation.

Table 5 summarizes the MuJoCo/Menagerie artifact tiers and acceptance evidence.

TABLE 5: MuJoCo/Menagerie-to-PyChrono artifact tiers.

Evidence tier	What Chrono Agentic built	Acceptance evidence
Asset and kinematic portability	86 portable MJCF robot entries; 83 with parsed joints for kinematic preview.	Source cards, screenshots, and explicit “no dynamics equivalence” scope.
MuJoCo-style dynamics	Cart-pole and guided hopper with dynamic bodies, joints, torque motors, support spring, gravity, and contact.	Bounded <code>-no-vis</code> : body/link counts, pole angle, pivot gap, contacts, torso height.
Robot dynamics abstractions	Two-link arm, skid-steer base, and stance-leg quadruped with PD motors, wheel torque, payload, spring-damper legs, and contact.	Bounded <code>-no-vis</code> : motor spans, torques, chassis velocity, foot contacts, trunk motion, leg force.

B. FROM VERIFIED DEMOS TO REUSABLE EVIDENCE

The reported transfer artifacts are evidential, not decorative. A transferred artifact becomes reusable evidence only when it has an accepted source file, source card, bounded execution evidence, visual review when relevant, mechanism-specific diagnostics, approximation notes, and a validation verdict.

This evidence standard changes how port quality is judged. A screenshot of a robot mesh supports visual asset portability; it does not certify torque response, contact behavior, or controller reproduction. A no-visualization diagnostic run is evidence only if it reports mechanism-specific signals, such as joint count, motor torque, tip position, contact count, yaw direction, foot contact, or constraint residual. Approximation notes are part of the artifact because they prevent later reuse from upgrading a semantic port into a false equivalence statement. Table 6 states the supported statement for each

witness.

TABLE 6: Transfer witness supported statements.

Witness	Chrono artifact	Supported statement
Spot asset	MJCF mesh, material, body-tree import	Portable visual/structural asset; no bit-exact MuJoCo dynamics claim.
Unitree Go2 asset	MJCF mesh, material, body-tree import	Robot asset portability witness with parsed joint metadata.
2R arm dynamics	Chrono bodies, revolute joints, motor, trajectory trace	Chrono-native torque response witness.
Skid-steer contact	Chrono chassis, wheels, contact material, bounded run	Contact/direction witness, not MuJoCo replay equivalence.

The reusable evidence record supports two downstream uses. For a human simulator developer, it records what source was read, what was preserved, what was approximated, and what passed. For simulator-specific training data, it provides supervised examples whose provenance includes a verified reconstruction and a declared scope, rather than only a prompt and answer.

VII. SCENARIO III: AUTO RESEARCH WITH CHRONO AGENTIC

The third scenario uses Chrono Agentic as a simulator-backed auto-research workflow. The platform turns a research question into a question-to-evidence pipeline: the agent formulates a subproblem, writes the PyChrono experiment, searches or identifies a parameter, checks convergence, repairs predicate mistakes, and stores the evidence trail.

The important distinction is between producing a plausible answer and producing a replayable research artifact. A plausible answer may quote the analytic pendulum threshold, fit a parameter by inspection, or report a torque value from one run. A replayable artifact must define the model, write the operational predicate, run the simulator over a declared horizon and timestep, store all probes, preserve the final witnesses, and state what numerical gate made the answer acceptable. Scenario III is built to exercise that sequence.

A. RESEARCH AND ARTIFACT CONTRACT

The research contract is the acceptance rule for every auto-research answer rather than a separate flowchart. An accepted artifact contains a question card, operational predicate, PyChrono model, search trace, witness replay, verification gate, and evidence record; a scalar answer by inspection is not accepted. The case sequence deliberately moves from analytic calibration to parameter identification and then to inverse design with no closed-form answer. Appendix A preserves the full visible/action trajectory at the level of research actions and verification gates rather than raw file names.

Normalized action trajectory. The trajectory in Scenario III is the external sequence of visible agent actions and accepted artifacts, not hidden model reasoning. It records what the user asked, how the task card constrained the experiment, what

TABLE 7: Normalized auto-research action trajectory. Each row is an externally checkable step that changes the research state or stores an artifact.

Step	External action	State change	Verification gate
T0	User request and task card	Three pendulum cases are normalized: inverse speed, damping identification, and torque-pulse swing-up. Constants, horizons, tolerances, and search variables are fixed before code generation.	Defines the experimental contract.
T1	PyChrono suite generation	The agent writes the model builder, simulator, force update, search routines, exporters, and static import check.	Establishes an executable simulator instrument.
T2	Predicate repair	“Reach the top” is rejected as ambiguous and split into horizontal reach, top-neighborhood reach, and finite-margin clear-top predicates.	Eq. (10) fixes the acceptance predicates before Case A is accepted.
T3	Case A execution	Bisection searches over v_0 store brackets, probes, final witness pairs, analytic comparison, residuals, and timestep sensitivity.	Analytic baseline plus residual and sensitivity gates.
T4	Case B identification	A synthetic trajectory with $c^* = 0.12$ is matched by first peak and peak-decay ratio, then replayed at the fitted value.	Parameter recovery and objective-collapse gate.
T5	Case C inverse design	A finite tangential pulse is bracketed and bisected on τ_0 from rest. The accepted run is the first-pass witness above the final failing probe.	Last-fail/first-pass witness gate.
T6	Evidence packaging	Results are normalized into manuscript tables, equations, predicates, search traces, repairs, convergence checks, witnesses, and stored records.	Connects computed results to paper-level evidence.

code was generated, where the predicate was repaired, which PyChrono probes were executed, why each search stopped, and what verification gate made the result acceptable. Table 7 is the compact version of that trace; the full normalized trace is preserved in Appendix Listing A.2.

The sequence exposes three failure modes. In Case A, the danger is conceptual: the top separatrix, a top-neighborhood

TABLE 8: Accepted auto-research pendulum cases.

Case	Research operation	Search predicate	Accepted result	Verification gate
A	Inverse-threshold calibration	v_0 for $\mathcal{P}_h$, $\mathcal{P}_{\text{top}}$, and $\mathcal{P}_{\text{clear}}$	4.4296 / 6.2628 / 6.2643 m s^{-1}	Energy baseline; length residual $< 2.0 \times 10^{-5}$ m.
B	Damping identification	c matching first peak p_1 and peak ratio ρ	$c = 0.12000$ N s m^{-1}	Synthetic truth $c = 0.120$; absolute error 2.96×10^{-10} .
C	Torque-pulse swing-up	τ_0 with $\mathcal{P}_{\text{clear}}$ and $T_p = 0.35$ s	$\tau_0 = 18.7386$ N m	Last fail/pass witnesses: 3.133 / 9.429 rad.

event, and a finite-margin clear-top event are not the same predicate. In Case B, the danger is identifiability by a weak statistic: matching one peak alone is less constraining than matching both the first peak and the peak-decay ratio. In Case C, the danger is a false monotonic or single-run claim: the torque threshold must be supported by a last-fail/first-pass bracket and a replay witness.

Two details make this trajectory different from a standalone demonstration script. First, the repair at T2 happens before accepting any Case A result: the agent does not let “reach the top” remain an informal visual phrase, but rewrites it into three executable predicates. Second, after T3–T5 the evidence is not just a final number. The trace records the bracket before each probe, the probe value, the predicate decision or objective values, the post-update width, and simulator diagnostics such as maximum height, maximum unwrapped angle, and distance-constraint residual.

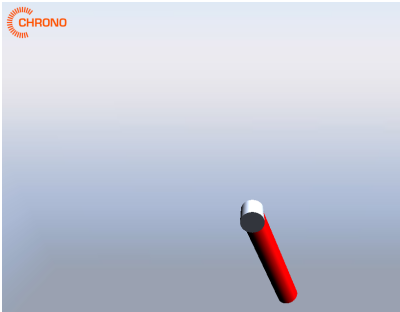


FIGURE 6: PyChrono/Irrlicht witness for the auto-research pendulum. The figure verifies the executable scene used for visual inspection; the numerical search evidence is reported in the case narratives and preserved in the trace appendix.

B. PYCHRONO MODEL AND THEORY

All three subproblems use a one-degree-of-freedom pendulum in PyChrono, with executable visual evidence shown in Figure 6. The visual run follows the official ChBody plus

ChLinkLockRevolute single-pendulum example, while the headless search suite uses a point bob constrained by ChLinkDistance. Both realize the constrained multibody form in Eq. (3), using the notation of MBD-NODE and FNODE-style constrained MBD studies [37], [38].

The visual and headless implementations have different purposes. The ChBody/ChLinkLockRevolute witness establishes that the user-facing PyChrono scene is an inspectable pendulum artifact with a visible pivot, body, and trajectory. The ChLinkDistance point-bob suite is the search instrument: it reduces geometry clutter, makes the distance residual easy to monitor, and allows the same code path to add damping or a torque-equivalent tangential load. Treating these as two views of the same constrained problem avoids confusing visual evidence with the numerical search mechanism. The shared contract is fixed across all cases: $m = 1$ kg, $L = 1$ m, $g = 9.81$ m/s^2 , default $\Delta t = 10^{-3}$ s, and search variables v_0 , c , and τ_0 for Cases A–C respectively.

At the multibody level, the experiment is a constrained DAE,

$$M(q)\ddot{q} = Q_g(q) + Q_u(q, \dot{q}, t) + \Phi_q(q)^\top \lambda, \quad \Phi(q) = 0, \quad (3)$$

where q are generalized coordinates, M is the mass matrix, Q_g is gravity, Q_u is an optional applied tangential load, Φ is the holonomic constraint, Φ_q is its Jacobian, and λ are constraint multipliers. In the headless point-bob implementation, Eq. (3) becomes

$$\begin{aligned} m\ddot{r} &= \begin{bmatrix} 0 \\ -mg \end{bmatrix} + f_t(r, \dot{r}, t)e_t(r) + \Phi_r(r)^\top \lambda, \\ r &= \begin{bmatrix} x \\ y \end{bmatrix}, \quad \Phi(r) = x^2 + y^2 - L^2 = 0. \end{aligned} \quad (4)$$

The PyChrono realization of Eq. (4) uses an NSC system, a spherical bob body, and a distance constraint for $\Phi(r) = 0$. The optional load is

$$\begin{aligned} e_t(r) &= \frac{1}{L} \begin{bmatrix} -y \\ x \end{bmatrix}, \\ v_t &= e_t(r)^\top \dot{r}, \\ f_t(r, \dot{r}, t) &= -c v_t + \frac{\tau_0}{L} \mathbf{1}_{0 \leq t \leq T_p}, \end{aligned} \quad (5)$$

implemented by ChLoadBodyForce. Reducing Eqs. (4) and (5) to the angular coordinate θ measured from the downward vertical gives

$$mL^2\ddot{\theta} + cL^2\dot{\theta} + mgL \sin \theta = \tau_0 \mathbf{1}_{0 \leq t \leq T_p}. \quad (6)$$

Equation (6) is the model behind the three cases: Case A sets $c = \tau_0 = 0$, Case B sets $\tau_0 = 0$ and searches c , and Case C sets $c = 0$ and searches τ_0 .

For Case A, the analytic baseline uses the undamped energy in Eq. (7), which gives the threshold speeds in Eq. (8):

$$E(\theta, \dot{\theta}) = \frac{1}{2}mL^2\dot{\theta}^2 + mgL(1 - \cos \theta), \quad (7)$$

$$v_0(y_*) = \sqrt{2g(y_* + L)}, \quad (8)$$

$$v_{\text{horizontal}} = \sqrt{2gL}, \quad v_{\text{top}} = 2\sqrt{gL}. \quad (9)$$

The experiment does not confuse the separatrix in Eq. (8) with a finite-time crossing event. The agent evaluates the operational predicates

$$\begin{aligned}\mathcal{P}_h(v_0) &= \mathbf{1} \left[\max_n y_n \geq 0 \right], \\ \mathcal{P}_{\text{top}}(v_0) &= \mathbf{1} \left[\max_n y_n \geq L - \varepsilon_y \right], \\ \mathcal{P}_{\text{clear}}(v_0) &= \mathbf{1} \left[\max_n \theta_n \geq \pi + \varepsilon_\theta \right],\end{aligned}\quad (10)$$

with $\varepsilon_y = 10^{-3}$ m and $\varepsilon_\theta = 0.02$ rad. Equation (10) is the contract used by the bisection code, not a post hoc interpretation of a plot.

C. SUBPROBLEM A: CALIBRATED INVERSE INITIAL SPEED

Case A calibrates the loop on an inverse problem with analytic checks: find the minimum initial speed for horizontal reach, top-neighborhood reach, and finite-margin top clearing. The first two predicates in Eq. (10) are compared with Eq. (8); the clear-top predicate is intentionally stronger because it requires finite-time crossing with margin.

The search variable is v_0 . For each predicate in Eq. (10), the agent brackets a false/true pair and applies the bisection update

$$\begin{aligned}m_k &= \frac{\ell_k + u_k}{2}, \\ (\ell_{k+1}, u_{k+1}) &= \begin{cases} (\ell_k, m_k), & \mathcal{P}(m_k) = 1, \\ (m_k, u_k), & \mathcal{P}(m_k) = 0, \end{cases}\end{aligned}\quad (11)$$

until $u_k - \ell_k \leq 2.0 \times 10^{-4}$ or the iteration limit is reached. Equation (11) is applied independently to $\mathcal{P}_h$, $\mathcal{P}_{\text{top}}$, and $\mathcal{P}_{\text{clear}}$.

The Case A trajectory has four visible actions. First, the horizontal-reach search brackets the event with a high probe $v_0 = 5.0939$ m/s, which succeeds with $\max y = 0.3225$, then bisections through failing probes at 2.5469 and 3.8204 m/s before localizing the final pair. The last failing witness is $v_0 = 4.429460$ m/s with $\max y = -4.97 \times 10^{-6}$, and the accepted witness is $v_0 = 4.429616$ m/s with $\max y = 6.52 \times 10^{-5}$. The final width is 1.55×10^{-4} m/s.

Second, the top-neighborhood search targets $\max y \geq L - 10^{-3}$, not a rotation event. The accepted value is $v_0 = 6.262755$ m/s with $\max y = 0.999039$, while the preceding false side remains below the height tolerance. Third, the clear-top search deliberately uses the stronger angle-margin predicate. It begins with a high pass at 6.7653 m/s and failures at 3.3827, 5.0740, and 5.9197 m/s before a pass at 6.3425 m/s. The final false/true witnesses are 6.264237 m/s with $\max \theta = 3.136$ and 6.264341 m/s with $\max \theta = 10.185$. These witnesses show that clear-top crossing is a different claim from near-top reach.

Fourth, the agent runs a timestep-sensitivity sweep after the three thresholds are accepted. The horizontal and top-neighborhood thresholds agree with Eq. (8) to within 1.69×10^{-4} and 1.37×10^{-4} m/s at $\Delta t = 10^{-3}$ s, and the top-threshold error falls from 7.87×10^{-3} to 3.69×10^{-5} m/s as Δt decreases from 10^{-2} to 5×10^{-4} s. The record contains

the predicate repair, bisection probes, analytic comparison, timestep sensitivity, and accepted threshold/convergence evidence.

D. SUBPROBLEM B: DAMPED PENDULUM IDENTIFICATION

Case B changes the task type from threshold recovery to parameter identification. A synthetic PyChrono run with $c^* = 0.12$ N s/m and initial speed 4 m/s produces $p_1^* = 1.334211$ rad and $\rho^* = 0.864557$. The agent recovers c with the self-consistency objective in Eq. (12) and the ternary update in Eq. (13):

$$J(c) = \left(\frac{p_1(c) - p_1^*}{p_1^*} \right)^2 + \left(\frac{\rho(c) - \rho^*}{\rho^*} \right)^2, \quad \rho(c) = \frac{p_2(c)}{p_1(c)}, \quad (12)$$

$$c_1 = \ell_k + \frac{u_k - \ell_k}{3}, \quad (13)$$

$$c_2 = u_k - \frac{u_k - \ell_k}{3}, \quad (14)$$

$$(\ell_{k+1}, u_{k+1}) = \begin{cases} (\ell_k, c_2), & J(c_1) < J(c_2), \\ (c_1, u_k), & J(c_1) \geq J(c_2). \end{cases} \quad (15)$$

Here p_1 is the first positive peak angle and $\rho = p_2/p_1$ is the peak-decay ratio. The action trajectory is again explicit. The agent first generates the target observation with $v_0 = 4.0$ m/s and $c^* = 0.12$ N s/m, obtaining $p_1^* = 1.334211$ rad and $\rho^* = 0.864557$. It then opens the search interval $c \in [0, 0.5]$ and records every ternary comparison. Iteration 1 evaluates $c = 0.1667$ and 0.3333 , finds objectives 2.92×10^{-3} and 4.82×10^{-2} , and keeps the left interval. Iteration 2 keeps the left interval again after $c = 0.1111$ gives a much smaller objective than 0.2222. Iteration 3 keeps the right subinterval, because $c = 0.1481$ is closer to the synthetic trajectory than 0.0741.

The middle of the trace shows the search narrowing around the true damping. By iteration 8, the interval is $[0.11065, 0.13992]$ and the probe $c = 0.120408$ has objective 2.41×10^{-7} . By iteration 14, the best tested value is $c = 0.120020$ with objective 6.02×10^{-10} . By iteration 26, the tested value $c = 0.12000008$ has objective 9.29×10^{-15} . The final iteration returns $c = 0.1200000003$ N s/m, interval width 1.76×10^{-9} , and objective 1.26×10^{-19} ; the fitted replay gives $p_1 = 1.3342110255$ and $\rho = 0.8645572208$. Case B is reported as a stored identification trajectory rather than as a scalar answer.

This case tests a different part of the agent. The source question is not “derive the damped pendulum equation” but “recover the damping used by a simulator trace.” Chrono Agentic must decide which observations make the identification well posed, write the PyChrono probe, choose a search range, store objective values, and stop only when the interval and fit are sufficient. Matching both p_1 and ρ is more constraining than matching a single peak: the first peak constrains the initial energy loss, while the ratio constrains decay over the next swing.

E. SUBPROBLEM C: TORQUE-PULSE SWING-UP

Case C removes the closed-form target. Starting from rest, the agent searches for the minimum torque pulse τ_0 in Eq. (5) that clears the top within an 8 s PyChrono run. The pulse lasts $T_p = 0.35$ s and is judged by $\mathcal{P}_{\text{clear}}$ in Eq. (10); Eq. (11) applies because the accepted bracket is monotone.

The torque trajectory begins by reusing the same `update_force` code path, but with damping set to zero and a tangential pulse active only for $0 \leq t \leq 0.35$ s. The first high probe $\tau_0 = 25.0$ N m succeeds with $\max \theta = 53.811$; the first midpoint $\tau_0 = 12.5$ N m fails with $\max \theta = 1.458$; the next midpoint $\tau_0 = 18.75$ N m succeeds with $\max \theta = 15.773$. At that point the trace has a genuine simulator-backed bracket $[12.5, 18.75]$ N m.

The narrowing sequence records why the final value is a threshold rather than a single favorable run. Probes at 15.625, 17.1875, 17.96875, 18.359375, 18.5546875, 18.65234375, 18.701171875, 18.7255859375, and 18.73779296875 N m all fail, with $\max \theta$ rising from 1.970 to 3.124 rad. The first near-threshold passes occur at 18.743896, 18.740845, 18.739319, and 18.738556 N m. The final false witness is $\tau_0 = 18.738365$ N m with $\max \theta = 3.133$ rad, below $\pi + 0.02$; the final true witness is $\tau_0 = 18.738556$ N m with $\max \theta = 9.429$ rad and $\max y = 1.000000000005$ m. The accepted bracket width is 1.91×10^{-4} N m.

The reported artifact in Case C is not the number alone. The accepted record includes a fail/pass witness pair because the result has no analytic reference comparable to Case A. The last failing probe demonstrates that a visually high swing is still below the clear-top predicate, while the first passing probe demonstrates that the pulse is sufficient under the same timestep, horizon, and load model. The inverse-design answer is supported by a search rule, a final bracket width, and replayable witnesses on both sides of the threshold.

F. ITERATION, CONVERGENCE, AND REPAIR

For the bisection cases, Eq. (11) halves the interval width after each predicate probe once the bracket is established. For the damping case, Eq. (13) contracts the interval by a factor of $2/3$ per iteration. The convergence records store the bracket before each probe, the probed value, the predicate decision or objective values, the interval width after the update, and the relevant simulation diagnostics. The distinguishing feature of the auto-research artifact is that the intermediate search path is inspectable in the normalized appendix trace without placing every probe row in the main text.

The numerical acceptance gate is applied before the final result is accepted. For each accepted run, the agent records

$$\begin{aligned} g_\Phi &= \max_n |\Phi(r_n)|, \\ w_K &= u_K - \ell_K, \\ e_{\Delta t} &= |v_{\Delta t} - v_{\text{ref}}|, \end{aligned} \quad (16)$$

where g_Φ is the maximum distance-constraint residual, w_K is the final search bracket width, and $e_{\Delta t}$ is timestep error against an analytic reference when one exists. The relevant repair was

conceptual: the first formulation conflated “reaches the top” with “clears the top,” so the agent split the top-neighborhood and clear-top predicates in Eq. (10). The timestep-sensitivity sweep shows that the calibrated inverse case is not a single-timestep artifact; the top-threshold error falls from 7.87×10^{-3} to 3.69×10^{-5} m/s as the timestep is refined.

G. EVIDENCE RECORD AND FUTURE QUESTIONS

The evidence unit for each case is the tuple summarized above: research question, PyChrono model, searched variable, predicate, result, and verification gate. The action trajectory is reported through two numbered tables and case narratives: Table 7 gives the T0–T6 agent actions, Table 8 gives the case-level accepted results, and the subproblem sections explain the search decisions, final witnesses, convergence behavior, and predicate repair. Listing A.2 preserves the full normalized trace.

The three cases form an initial auto-research suite: analytic calibration, parameter identification, and inverse design. Natural extensions include friction, spring-damper, beam-stiffness, slider-crank, vehicle-braking, and double-pendulum inverse problems; each still needs a simulator-backed predicate, search space, verification gate, and evidence record.

The auto-research scenario also clarifies the division of labor between platform execution and domain judgment. Chrono Agentic formulates and executes simulator-backed subproblems, while domain expertise remains responsible for judging scientific importance, model adequacy for a real system, and identifiability from the chosen observations. The contribution is an explicit, replayable, and checkable computational path from question to simulator-backed answer.

VIII. DISCUSSION AND VALIDITY

Across the three scenarios, every reported result names the artifact, the evidence, and the supported scope. Chrono Agentic adds typed memory, selected PyChrono skills, execution, visual/physics review, repair, and clean-source delivery around a frozen base model. In Scenario I, language, image, or edit intent becomes runnable PyChrono source with execution and review evidence; top-down, wide-site, and trench-side construction views remain visual-review records rather than score, execution-rate, or physics-validity claims. In Scenario II, EXUDYN and MuJoCo/Menagerie material becomes inspectable PyChrono artifacts with selected Chrono-native dynamics witnesses and explicit equivalence notes. In Scenario III, an auto-research question becomes equations, predicates, PyChrono probes, search traces, repairs, convergence checks, and stored records.

This platform complements training-time adaptation. ChronoLLM improves the model prior for PyChrono code, while Chrono Agentic improves the artifact-generation process through inference-time orchestration, execution feedback, visual/physics review, and repair [2]. The output is a traceable artifact record rather than a source file alone. The reported evidence is tied to PyChrono 9.0.0, the reported transfer

artifacts, the pendulum suite, and the available orchestration components.

A key validity risk is evidence selection. Visual captures, diagnostic traces, and benchmark scores are generated by scripts that must themselves be trusted. The mitigation is separation: the clean source, review evidence, and result records are stored separately, and scalar claims point to stated verification gates rather than screenshots alone. The reported artifacts cover multibody, robot, transfer, and pendulum-research tasks. Extending the platform to additional Chrono modules, source simulators, and user-interface modalities is future evaluation work.

Benchmark interpretation is a separate risk. SimBench provides a controlled text-only benchmark score, but it cannot score the entire artifact record. A system that produces stronger review evidence may not always receive a higher Ref-Doc score, and a high Ref-Doc score does not prove visual or physical adequacy. Chrono Agentic treats benchmark scoring as one evidence channel among several.

The evaluation operates at three levels. At the source level, a reader can inspect whether the PyChrono program uses the intended bodies, constraints, loads, sensors, and solver/timestep choices. At the run level, a reader can replay bounded no-visualization commands or visual captures and check that diagnostics are finite and mechanism-specific. At the evidence level, a reader can connect each paper statement to the stored artifact record. Statements that rely on source, run, and evidence claims are attached to all three levels. Specialized results, such as asset portability without dynamics equivalence, are stated with an explicit scope.

EXUDYN and MuJoCo/Menagerie transfer is treated as semantic reconstruction, not exact simulator equivalence. A cross-simulator artifact can preserve a mechanism, a geometry hierarchy, a control phase, or a diagnostic, while still using a different contact model, solver, constraint stabilization method, or actuator abstraction. This limitation also makes reuse safer: a later reader can distinguish a visual asset witness, a Chrono-native dynamics witness, and a source-card record.

The auto-research suite reports a platform workflow for simulator-backed question answering. The agent handles formulation, implementation, search, repair, and record keeping for research questions expressed against a PyChrono model. Domain scientists still decide whether the model is sufficient for a real device, whether a chosen observation set is the best possible identification design, or whether a torque pulse is physically optimal under hardware constraints not present in the model. Chrono Agentic's contribution is that the computational path from question to simulator-backed answer is inspectable.

IX. CONCLUSION

Chrono Agentic is an inference-time orchestration platform for building executable PyChrono simulations. The three pipeline scenarios show the platform across three input classes: natural-language, image, or edit intent becomes simulation code; EXUDYN and MuJoCo-style material becomes PyChrono recon-

struction artifacts; and research questions become simulator-backed equations, predicates, probes, searches, repairs, and stored result records. On SimBench, the platform improves average Ref-Doc score from 50.60 to 54.12 over direct single-LLM generation in the matched 204-row evaluation. Scenario I preserves top-down, wide-site, and trench-side construction views for visual review while keeping benchmark and physics statements tied to source and run evidence. The contribution is a simulator-agent platform with a concrete protocol: construct, run, inspect, repair, document, and deliver clean executable artifacts with inspectable evidence. Accepted simulations are delivered in a form that simulator experts can inspect and reproduce.

APPENDIX A ARTIFACT LISTINGS

The appendix records two compact audit artifacts: a representative Scenario II source excerpt and a path-free Scenario III auto-research trace. These listings show the structure of accepted artifacts without turning the appendix into an inventory.

a: EXUDYN-to-PyChrono trace.

The artifacts in Figure 4 are tied to saved visible dialogue, concrete PyChrono sources, and regenerated captures. The preserved session contains 2,018 visible messages and 164 accepted checkpoints; the exported audit record excludes hidden instructions, raw tool payloads, and hidden reasoning. Listing A.1 gives the source pattern for one accepted port: constants, helper geometry, Chrono body construction, and mechanism-specific diagnostics.

Listing A.1. Representative PyChrono excerpt: bearing shaft with two ball bearings.

```

import math

import pychrono.core as chrono

# Representative excerpt from the accepted EXUDYN-to-PyChrono bearing-shaft
# port.
# Nonessential setup and batch-run helpers are omitted from this listing.

OUTSIDE_DIAMETER = 0.080
BORE_DIAMETER = 0.050
WIDTH = 0.016
N_BALLS = 14
RADIUS_CAGE = 0.0325
RADIUS_BALL = 0.00873 / 2.0
SHAFT_RADIUS = 0.5 * BORE_DIAMETER
SHAFT_LENGTH = 0.120
OMEGA_SHAFT = 2.0 * math.pi * 2.0
CAGE_SPEED_RATIO = 0.42
STEP = 1e-3

def color(r, g, b):
    return chrono.ChColor(r, g, b)

def add_ring_beads(body, ring_radius, bead_radius, z_offset, tint, count):
    for i in range(count):
        angle = 2.0 * math.pi * i / count
        point = chrono.ChVector3d(
            ring_radius * math.cos(angle),
            ring_radius * math.sin(angle),
            z_offset,
        )
        shape = chrono.ChVisualShapeSphere(bead_radius)
        shape.SetColor(tint)
        body.AddVisualShape(shape, chrono.ChFramed(point))

def build_system():
    system = chrono.ChSystemNSC()
    system.SetGravitationalAcceleration(chrono.ChVector3d(0, -9.81, 0))

    ground = chrono.ChBody()
    ground.SetName("bearing ground")
    ground.SetFixed(True)
    ground.EnableCollision(False)
    system.AddBody(ground)

```

```

shaft = chrono.ChBodyEasyCylinder(
    chrono.ChAxis_Z, SHAFT_RADIUS, SHAFT_LENGTH, 7800, True, False
)
shaft.SetName("rotating bearing shaft and inner rings")
shaft.SetFixed(True)
shaft.GetVisualShape(0).SetColor(color(0.12, 0.42, 0.85))
system.AddBody(shaft)

cages = []
balls = []
bearing_offsets = [-0.3 * SHAFT_LENGTH, 0.3 * SHAFT_LENGTH]
for bearing_index, z_offset in enumerate(bearing_offsets):
    outer = chrono.ChBody()
    outer.SetName(f"bearing outer bearing ring {bearing_index + 1}")
    outer.SetFixed(True)
    outer.SetPos(chrono.ChVector3d(0, 0, z_offset))
    add_ring_beads(outer, 0.5 * OUTSIDE_DIAMETER, 0.0018, 0, color(0.4,
65 0.4, 0.42), 56)
    system.AddBody(outer)

    cage = chrono.ChBody()
    cage.SetName(f"bearing cage {bearing_index + 1}")
    cage.SetFixed(True)
    cage.SetPos(chrono.ChVector3d(0, 0, z_offset))
    add_ring_beads(cage, RADIUS_CAGE, 0.0012, 0, color(0.95, 0.64, 0.16),
70 N_BALLS)
    system.AddBody(cage)
    cages.append(cage)

    bearing_balls = []
    for i in range(N_BALLS):
        ball = chrono.ChBodyEasySphere(RADIUS_BALL, 7800, True, False)
        ball.SetName(f"bearing {bearing_index + 1} ball {i + 1:02d}")
        ball.SetFixed(True)
        ball.GetVisualShape(0).SetColor(color(0.88, 0.88, 0.86))
        system.AddBody(ball)
        bearing_balls.append(ball)
        balls.append(bearing_balls)

    system_bearing_items = {"shaft": shaft, "cages": cages, "balls": balls, "
85 offsets": bearing_offsets}
    update_bearing_kinematics(system)
    return system

def update_bearing_kinematics(system):
    items = system_bearing_items
    time = system.GetChTime()
    shaft_angle = OMEGA_SHAFT * time
    cage_angle = CAGE_SPEED_RATIO * shaft_angle
    ball_spin = -OMEGA_SHAFT * SHAFT_RADIUS / RADIUS_BALL

    items["shaft"].SetRot(chrono.QuatFromAngleZ(shaft_angle))
    items["shaft"].SetAngVelParent(chrono.ChVector3d(0, 0, OMEGA_SHAFT))

    for bearing_index, z_offset in enumerate(items["offsets"]):
        items["cages"][bearing_index].SetRot(chrono.QuatFromAngleZ(cage_angle)
90 )
        for i, ball in enumerate(items["balls"][bearing_index]):
            angle = cage_angle + 2.0 * math.pi * i / N_BALLS
            position = chrono.ChVector3d(
                RADIUS_CAGE * math.cos(angle),
                RADIUS_CAGE * math.sin(angle),
                z_offset,
            )
            tangent = chrono.ChVector3d(-math.sin(angle), math.cos(angle), 0)
            ball.SetPos(position)
            ball.SetRot(chrono.QuatFromAngleAxis(ball_spin * time, tangent))

100

def simulate_no_vis(duration=2.0, step=STEP):
    system = build_system()
    while system.GetChTime() < duration:
        update_bearing_kinematics(system)
        system.DoStepDynamics(step)
120 return system

```

b: Auto-research trajectory.

The visible/action trajectory is summarized in Table 7. It records external actions rather than hidden model reasoning: the user request, task-card constraints, PyChrono implementation step, predicate repair before Case A acceptance, Case A/B/C search execution, and final recording of the results reported here. Listing A.2 gives the path-free normalized trace and omits hidden instructions, hidden model reasoning, raw tool payloads, and implementation-only inventories.

Listing A.2. Path-free normalized visible/action trajectory for the auto-research pendulum suite.

```

T0. USER QUESTION AND RESEARCH CONTRACT
- Input: formulate a simulator-backed pendulum research example, not a
single demonstration animation.
- Accepted artifact contract: question card, PyChrono model, operational
predicate, search trace, witness replay, convergence gate, and a bounded
claim statement.
- Initial risk: a plausible analytic answer could bypass the simulator, and
a plausible animation could bypass the numerical predicate.
- Decision: use the pendulum because it has an analytic calibration case,

```

an identifiable damping case, and an inverse-design torque case.

T1. MODEL SELECTION AND PYCHRONO IMPLEMENTATION

- Build two compatible views of the same constrained pendulum.
- Visual witness view: visible pivot, bob, link, and trajectory in a PyChrono/Irrlicht scene for human inspection.
- Headless probe view: point bob, distance constraint, gravity, optional tangential load, fixed time horizon, fixed default timestep, and recorded diagnostic signals.
- Shared constants: mass 1 kg, length 1 m, gravity 9.81 m/s².
- Required diagnostics: maximum height, maximum unwrapped angle, final bracket width, distance-constraint residual, and pass/fail predicate.
- Acceptance rule: no result is accepted from a scalar value alone; every reported value must be supported by at least one replayed witness or a final fail/pass bracket.

T2. PREDICATE REPAIR BEFORE CASE A

- Initial ambiguous phrasing: "reach the top" was not sufficient.
- Repair: split the physical question into three executable predicates.
- Predicate A1, horizontal reach: pass when maximum bob height is at least the pivot height.
- Predicate A2, top-neighborhood reach: pass when maximum bob height is within 1 mm of the top height.
- Predicate A3, clear-top event: pass only when the unwrapped angle exceeds pi plus a 0.02 rad margin during the finite-time run.
- Consequence: the near-top threshold and the finite-time clear-top threshold are treated as different claims.

T3. CASE A, INVERSE INITIAL SPEED WITH ANALYTIC CHECKS

- Research question: find the minimum initial tangential speed for the three predicates in T2.
- Search method: bracket a false and true value, then apply bisection until the velocity bracket is no wider than 2.0e-4 m/s.
- Analytic baseline: horizontal reach should occur near sqrt(2 g L); the separatrix top-height threshold should occur near 2 sqrt(g L).
- Horizontal search:
 - A high probe at 5.0939 m/s passed with maximum height 0.3225 m.
 - Lower probes at 2.5469 and 3.8204 m/s failed.
 - Final false witness: 4.429460 m/s with maximum height just below the pivot level.
 - Final true witness: 4.429616 m/s with maximum height just above the pivot level.
 - Accepted width: 1.55e-4 m/s.
- Top-neighborhood search:
 - Target: maximum height at least L minus 1 mm.
 - Accepted value: 6.262755 m/s, maximum height 0.999039 m.
 - The false side remains below the height tolerance.
- Clear-top search:
 - Target: finite-time angle exceeds pi plus 0.02 rad.
 - Early probes below the top event failed even when the swing was visually high.
 - Final false witness: 6.264237 m/s, maximum angle 3.136 rad.
 - Final true witness: 6.264341 m/s, maximum angle 10.185 rad.
 - Interpretation: clear-top crossing is not the same evidence as near-top height.
- Convergence check:
 - The analytic height thresholds match the simulator thresholds to the reported tolerance at the default timestep.
 - Timestep refinement reduces the top-threshold error from 7.87e-3 m/s to 3.69e-5 m/s.
- Case A accepted because the predicates were repaired, the bisection brackets closed, the analytic checks agreed, and timestep sensitivity was recorded.

T4. CASE B, DAMPED-PENDULUM IDENTIFICATION

- Research question: recover the damping coefficient used to generate a synthetic damped-pendulum trace.
- Target generation: use damping c = 0.12 N s/m and initial speed 4 m/s.
- Observations: first positive peak angle and the ratio between the second and first positive peaks.
- Objective: minimize the squared relative mismatch in both the first peak and the peak-decay ratio.
- Search method: ternary search over damping because the self-consistency objective is unimodal over the chosen interval.
- Early trajectory:
 - Iteration 1 compared 0.1667 and 0.3333 N s/m and kept the lower side.
 - Iteration 2 compared 0.1111 and 0.2222 N s/m and again kept the lower side.
 - Iteration 3 kept the side containing 0.1481 N s/m over 0.0741 N s/m.
- Lock-on trajectory:
 - By iteration 8, the active interval contained 0.120408 N s/m with objective 2.41e-7.
 - By iteration 14, the best tested value was 0.120020 N s/m with objective 6.02e-10.
 - By iteration 26, the tested value 0.12000008 N s/m had objective 9.29e-15.
- Final accepted result:
 - Damping estimate: 0.1200000003 N s/m.
 - Final interval width: 1.76e-9 N s/m.
 - Final objective: 1.26e-19.
 - Replay peaks match the synthetic first peak and decay ratio.
- Case B accepted because the recovered coefficient is supported by the search path, the final interval, and a replay against both target observables.

T5. CASE C, TORQUE-PULSE SWING-UP

- Research question: find the minimum torque-pulse amplitude that clears the top from rest.
- Model change: disable damping and apply a tangential torque-equivalent load only during the first 0.35 s of the run.
- Predicate: the same clear-top event from T2, so a visually high swing is still a failure unless the angle margin is crossed.
- Search method: bisection on pulse amplitude using the finite-time pass/fail predicate.
- Bracketing:
 - 25.0 N m passed with maximum angle 53.811 rad.
 - 12.5 N m failed with maximum angle 1.458 rad.
 - 18.75 N m passed with maximum angle 15.773 rad.
- Narrowing:
 - Probes at 15.625, 17.1875, 17.96875, 18.359375, 18.5546875, 18.65234375, 18.701171875, 18.7255859375, and 18.73779296875 N m

```

all failed.
- First near-threshold passing probes occurred just above those failures.
- Final accepted bracket:
- Last false witness: 18.738365 N m, maximum angle 3.133 rad.
- First true witness: 18.738556 N m, maximum angle 9.429 rad.
- Final bracket width: 1.91e-4 N m.
- Case C accepted because the answer has no analytic shortcut and is instead
  supported by a last-fail/first-pass simulator bracket under a declared
  timestep, horizon, and load model.

```

T6. PAPER-LEVEL PACKAGING AND EVIDENCE SCOPE

```

- Main text reports the equations, predicates, search rules, case narratives,
  representative probe excerpts, and convergence summaries.
- The appendix preserves this normalized action trace as an audit record of
  visible actions and accepted artifacts.
- Hidden model reasoning, hidden instructions, raw tool payloads, and local
  file inventories are excluded from the manuscript.
- Platform claim: the auto-research suite demonstrates a reproducible
  question-to-evidence workflow for simulator-backed research questions;
  domain judgment remains responsible for real-system interpretation and
  scientific relevance.

```

REFERENCES

- [1] Project Chrono, "Project Chrono - An Open-Source Physics Engine," <https://projectchrono.org/>, 2026.
- [2] J. Wang, A. Negrut, H. Zhang, K. Slaton, S. Wang, R. Serban, J. Wu, and D. Negrut, "Chronollm: customizing language models for physics-based simulation code generation," *Multibody System Dynamics*, pp. 1–45, 2026. [Online]. Available: <https://doi.org/10.1007/s11044-026-10152-x>
- [3] H. Wang, J. Wang, B. Zou, R. Serban, and D. Negrut, "Coding agent is good as world simulator," 2026, arXiv:2605.14398. [Online]. Available: <https://arxiv.org/abs/2605.14398>
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [6] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023. [Online]. Available: <https://arxiv.org/abs/2303.17651>
- [7] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better llm agents," in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01030>
- [8] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=mXpq6ut8J3>
- [9] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," *arXiv preprint arXiv:2404.05427*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.05427>
- [10] D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/2de5d16682c3c35007e4e92982f1a2ba-Paper.pdf
- [11] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International conference on machine learning*. PMLR, 2019, pp. 2555–2565.
- [12] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*, 2020.
- [13] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse domains through world models," *arXiv preprint arXiv:2301.04104*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.04104>
- [14] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps *et al.*, "Genie: Generative interactive environments," in *Forty-first International Conference on Machine Learning*, 2024.
- [15] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding *et al.*, "Cosmos world foundation model platform for physical ai," *arXiv preprint arXiv:2501.03575*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.03575>
- [16] H. Bansal, Z. Lin, T. Xie, Z. Zong, M. Yarom, Y. Bitton, C. Jiang, Y. Sun, K.-W. Chang, and A. Grover, "Videophy: Evaluating physical commonsense for video generation," *arXiv preprint arXiv:2406.03520*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.03520>
- [17] D. Li, Y. Fang, Y. Chen, S. Yang, S. Cao, J. Wong, M. Luo, X. Wang, H. Yin, J. E. Gonzalez, I. Stoica, S. Han, and Y. Lu, "Worldmodelbench: Judging video generation models as world models," *arXiv preprint arXiv:2502.20694*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.20694>
- [18] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.
- [19] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.10470>
- [20] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang *et al.*, "Sapien: A simulated part-based interactive environment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 097–11 107.
- [21] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. S. Chaplot, O. Maksymets *et al.*, "Habitat 2.0: Training home assistants to rearrange their habitat," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 251–266.
- [22] C. Li, F. Xia, R. Martin-Martin, M. Lingelbach, S. Srivastava, B. Shen, K. E. Vainio, C. Gokmen, G. Dharan, T. Jain *et al.*, "igibson 2.0: Object-centric simulation for robot learning of everyday household tasks," in *Conference on Robot Learning*. PMLR, 2022, pp. 455–465.
- [23] M. Deitke, E. VanderBilt, A. Herrasti, L. Weihs, K. Ehsani, J. Salvador, W. Han, E. Kolve, A. Kembhavi, and R. Mottaghi, "Proctor: Large-scale embodied ai using procedural generation," *Advances in Neural Information Processing Systems*, vol. 35, pp. 5982–5994, 2022.
- [24] Y. Yang, F. Zhao, Y. Zhu, P. Zhang, X. Chen, and S. Huang, "Holodeck: Language guided generation of 3d embodied ai environments," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [25] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [26] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations*, 2024.
- [27] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, "Openhands: An open platform for ai software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.16741>
- [28] Y. Wang, H. Yang, M. Guo, X. Qiu, T.-H. Wang, W. Matusik, J. B. Tenenbaum, and C. Gan, "Physcensis: Physics-augmented llm agents for complex physical scene arrangement," *arXiv preprint arXiv:2602.14968*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.14968>
- [29] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.12931>
- [30] R. G. Sargent, "Verification and validation of simulation models," in *Proceedings of the 2011 Winter Simulation Conference*. IEEE, 2011, pp. 183–198. [Online]. Available: <https://doi.org/10.1109/WSC.2011.6147750>
- [31] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. [Online]. Available: <https://doi.org/10.1145/3458723>
- [32] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019, pp. 220–229.
- [33] G. Comanici, E. Bieber, M. Schaeckermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context,

- and next generation agentic capabilities,” *arXiv preprint arXiv:2507.06261*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.06261>
- [34] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan, “Fleming: a visual language model for few-shot learning,” in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: <https://openreview.net/forum?id=EbMuimAbPbs>
- [35] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *Advances in neural information processing systems*, vol. 36, pp. 34 892–34 916, 2023.
- [36] J. Wang, A. Negrut, H. Wang, H. Zhang, and D. Negrut, “Simbench: A framework for evaluating and diagnosing llm-based digital-twin generation for multi-physics simulation,” *IEEE Access*, vol. 14, pp. 61 784–61 808, 2026.
- [37] J. Wang, S. Wang, H. M. Unjhwala, J. Wu, and D. Negrut, “MBD-NODE: physics-informed data-driven modeling and simulation of constrained multibody systems,” *Multibody System Dynamics*, vol. 64, pp. 543–585, 2025. [Online]. Available: <https://doi.org/10.1007/s11044-024-10012-6>
- [38] H. Wang, J. Wang, and D. Negrut, “FNODE: Flow-matching for data-driven simulation of constrained multibody systems,” 2025, arXiv:2509.00183. [Online]. Available: <https://arxiv.org/abs/2509.00183>



JINGQUAN WANG received the B.S. degree in mathematics from the University of Science and Technology of China, Hefei, China, in 2020. He is currently pursuing the Ph.D. degree in mechanical engineering at the University of Wisconsin–Madison, Madison, WI, USA. His research interests include digital twins, multi-physics simulation, simulator-code generation, and LLM-assisted scientific computing.



HONGYU WANG received the B.E. degree in electronic information engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2024, through a joint educational program with the University of Glasgow. He is currently pursuing the M.S. degree in electrical and computer engineering at the University of Wisconsin–Madison, Madison, WI, USA. His research interests include LLM-based code generation for digital twins, multibody dynamics, and data-driven simulation.



ANDREW NEGRUT is currently pursuing the B.S. degree in computer science at Rice University, Houston, TX, USA. His research interests include large language models, fine-tuning, retrieval-augmented generation, and simulator-oriented code generation.

ASHVIN ANILKUMAR is with the University of Wisconsin–Madison, Madison, WI, USA. His research interests include simulation tooling, PyChrono artifact construction, and LLM-assisted workflows for executable simulation.

RADU SERBAN is with the University of Wisconsin–Madison, Madison, WI, USA. His research interests include multibody dynamics, high-performance simulation, vehicle and robot simulation, and scientific software for physics-based modeling.



DAN NEGRUT is the Bernard A. and Frances M. Weideman Professor in the Department of Mechanical Engineering at the University of Wisconsin–Madison, Madison, WI, USA. He has courtesy appointments in the Department of Computer Sciences and the Department of Electrical and Computer Engineering. He received the Ph.D. degree in mechanical engineering from the University of Iowa in 1998. Before joining the University of Wisconsin–Madison in 2005, he worked for Mechanical Dynamics, Inc., served as an adjunct assistant professor at the University of Michigan, and was a visiting scientist at Argonne National Laboratory. His research interests include high-performance computing, computational dynamics, artificial intelligence, terramechanics, autonomous vehicles, robotics, and fluid-solid interaction. He received the NSF CAREER Award in 2009. He is an NVIDIA CUDA Fellow and an ASME Fellow.

...